Namespaces and Shadowing

Compiler Sessional Online

Problem Statement

Task 1

Create a command called `N` for namespaces. This command takes the name of the namespace as argument, and switches the context to that namespace. For example -

```
N std
```

Switches the context to a namespace named `std`. If there is no such namespace, the `N` command first creates the namespace and then switches to it. Originally, the context is in the `default` namespace.

Every namespace will have their own symbol table. All operations are carried out only in the current namespace.

Hint

Simply maintain many symbol tables with different `name`, and switch according to their `name`.

Task 2

Create a command called `SH` for printing currently shadowed variables.
Shadowed Variable: A variable is shadowed, if some other variable in a descendant scope shares the same name.

In this assignment, you have to print such shadowed variables along with their shadower variable (the variable in the descendant scope that caused the shadowing). Consider the following example -

```
1  // ScopeTable# 1
2  int a;
3  int b;
4  {
5    // ScopeTable# 2
6    int a;
7    int b;
8    {
9      // ScopeTable# 3
10     int a;
11     ----> We want to get the shadowing report at this point
12   }
13 }
```

The shadowing report at line 11 should be -

```
"a" from ScopeTable# 3 shadows "a" from ScopeTable# 2
"a" from ScopeTable# 3 shadows "a" from ScopeTable# 1
"b" from ScopeTable# 2 shadows "b" from ScopeTable# 1
```

Remember, the shadower variable is the one the programmer ends up accessing.

Also, a variable only shadows other variables **in its own namespace**. It cannot shadow other namespace variables.

Hint

Search for shadower-shadowed pairs in the same symbol table

Restrictions

- You **can** modify existing classes from your practice assignment
- You **can** use STL
- Must compile with `-fsanitize=address`

Sample input, output, and explanations are on the following pages.

Examples

Sample Input

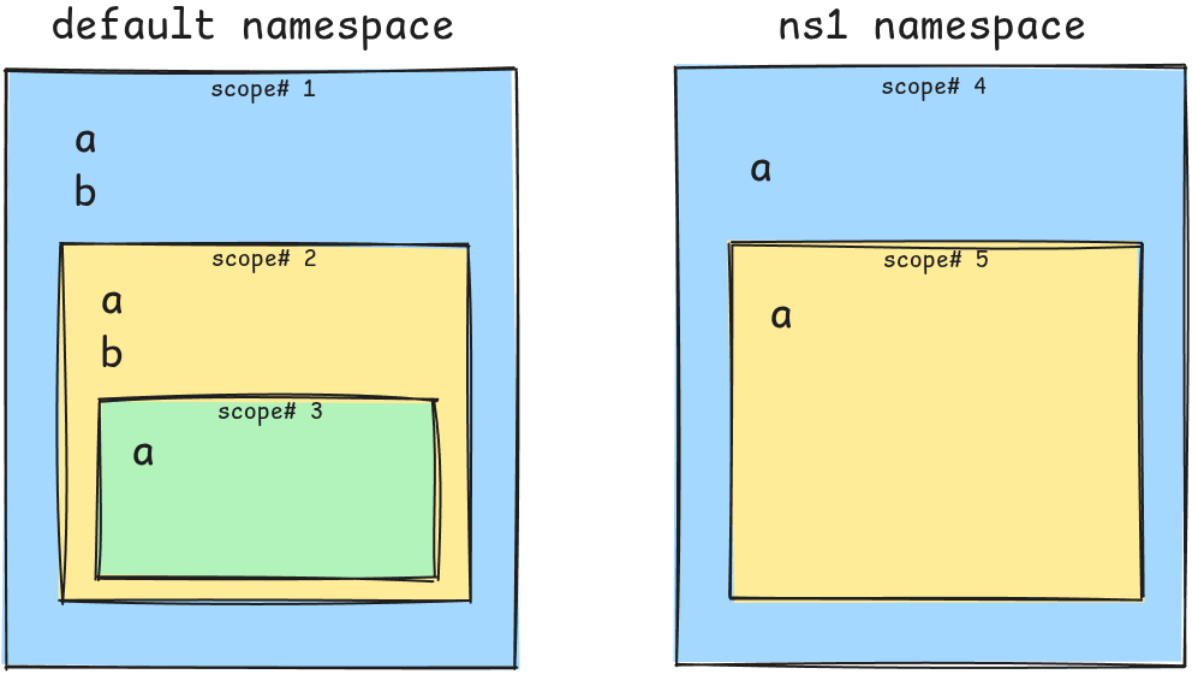
```
3
I a VAR
I b VAR
S
I a VAR
I b VAR
S
I a VAR
SH
N ns1
I a VAR
S
I a VAR
SH
L b
N default
L b
Q
```

Sample Output

```
 ScopeTable# 1 created
Cmd 1: I a VAR
 Inserted in ScopeTable# 1 at position 2, 1
Cmd 2: I b VAR
 Inserted in ScopeTable# 1 at position 3, 1
Cmd 3: S
 ScopeTable# 2 created
Cmd 4: I a VAR
 Inserted in ScopeTable# 2 at position 2, 1
Cmd 5: I b VAR
 Inserted in ScopeTable# 2 at position 3, 1
Cmd 6: S
 ScopeTable# 3 created
Cmd 7: I a VAR
 Inserted in ScopeTable# 3 at position 2, 1
Cmd 8: SH
 "a" from ScopeTable# 3 shadows "a" from ScopeTable# 2
 "a" from ScopeTable# 3 shadows "a" from ScopeTable# 1
 "b" from ScopeTable# 2 shadows "b" from ScopeTable# 1
Cmd 9: N ns1
 Namespace 'ns1' created
Cmd 10: I a VAR
 Inserted in ScopeTable# 4 at position 2, 1
Cmd 11: S
 ScopeTable# 5 created
Cmd 12: I a VAR
 Inserted in ScopeTable# 5 at position 2, 1
Cmd 13: SH
 "a" from ScopeTable# 5 shadows "a" from ScopeTable# 4
Cmd 14: L b
 'b' not found in any of the ScopeTables
Cmd 15: N default
 Switched to namespace 'default'
Cmd 16: L b
 'b' found in ScopeTable# 2 at position 3, 1
Cmd 17: Q
 ScopeTable# 3 removed
 ScopeTable# 2 removed
 ScopeTable# 1 removed
 ScopeTable# 5 removed
 ScopeTable# 4 removed
```

Explanation

The sample input depicts the following scenario



Marks Distribution

Task	Marks
Namespaces	5
Shadowing	5
Total	**10**